
TEORÍA DE BASE DE DATOS ÍNDICES

Prof. Sergio Salinas

TRABAJO PRÁCTICO

1 Introducción

Los índices en PostgreSQL permiten mejorar el rendimiento de las consultas, reduciendo la cantidad de datos que deben recorrerse.

2 Índice B-Tree

Es el tipo de índice por defecto. Utiliza un árbol balanceado y es eficiente para búsquedas por igualdad y por rango. Es el más utilizado en la práctica. **B-Tree:** <https://www.youtube.com/watch?v=K1a2Bk8NrYQ>

```
1 CREATE INDEX idx_nombre ON tabla(columna);
```

3 Índice Hash

Optimizado únicamente para comparaciones de igualdad (=). No sirve para búsquedas por rango ni ordenamiento.

```
1 CREATE INDEX idx_hash ON tabla USING HASH(columna);
```

4 Índice GiST

Es un tipo de índice flexible que permite trabajar con distintos tipos de datos complejos, como geometría o texto. Se usa mucho con extensiones.

```
1 CREATE INDEX idx_gist ON tabla USING GIST(columna);
```

5 Índice SP-GiST

Variante de GiST optimizada para estructuras de datos particionadas, como árboles o direcciones IP.

```
1 CREATE INDEX idx_spgist ON tabla USING SPGIST(columna);
```

6 Índice GIN

Ideal para columnas que contienen múltiples valores, como arrays o JSON. Muy utilizado en búsquedas de texto completo.

```
1 CREATE INDEX idx_gin ON tabla USING GIN(columna);
```

7 Índice BRIN

Indexa rangos de bloques en lugar de filas individuales. Es muy eficiente en tablas grandes donde los datos están ordenados (por ejemplo, fechas).

```
1 CREATE INDEX idx_brin ON tabla USING BRIN(columna);
```

8 Índices Multicolumna

Permiten indexar más de una columna. Son útiles cuando las consultas filtran por varias columnas al mismo tiempo.

```
1 CREATE INDEX idx_multi ON tabla(col1, col2);
```

9 Índices Parciales

Indexan solo un subconjunto de filas que cumplen una condición. Reducen el tamaño del índice y mejoran el rendimiento en consultas específicas.

```
1 CREATE INDEX idx_parcial ON tabla(columna)
2 WHERE estado = 'activo';
```

10 Índices Funcionales

Indexan el resultado de una expresión o función. Son útiles cuando las consultas aplican funciones sobre columnas.

```
1 CREATE INDEX idx_lower ON tabla(LOWER(nombre));
```

11 Índices Únicos

Garantizan que no existan valores duplicados en una columna o conjunto de columnas.

```
1 CREATE UNIQUE INDEX idx_unico ON tabla(columna);
```

12 PARTE I

Ejercicio 1: Búsqueda básica

Creación de tabla

```
CREATE TABLE clientes (  
    id SERIAL PRIMARY KEY,  
    nombre TEXT,  
    email TEXT,  
    ciudad TEXT  
);
```

Carga de datos

```
1 INSERT INTO clientes (nombre, email, ciudad)  
2 SELECT  
3     'Cliente ' || i,  
4     'cliente' || i || '@mail.com',  
5     CASE WHEN i % 5 = 0 THEN 'Mendoza' ELSE 'Otra' END  
6 FROM generate_series(1, 100000) i;
```

Consulta sin índice

```
1 EXPLAIN ANALYZE  
2 SELECT * FROM clientes  
3 WHERE email = 'cliente50000@mail.com';
```

Creación de índice

```
1 CREATE INDEX idx_clientes_email  
2 ON clientes(email);
```

Consulta con índice

```
1 EXPLAIN ANALYZE  
2 SELECT * FROM clientes  
3 WHERE email = 'cliente50000@mail.com';
```

Preguntas:

- ¿Qué cambia en el plan de ejecución?
- ¿Qué tipo de scan aparece?

Ejercicio 2: Índices y filtros

```
1 EXPLAIN ANALYZE  
2 SELECT * FROM clientes  
3 WHERE ciudad = 'Mendoza';
```

```
1 CREATE INDEX idx_clientes_ciudad
2 ON clientes(ciudad);
```

Pregunta: ¿Conviene indexar columnas con pocos valores distintos?

Ejercicio 3: Índices compuestos

```
1 CREATE TABLE ventas (
2     id SERIAL PRIMARY KEY,
3     cliente_id INT,
4     fecha DATE,
5     total NUMERIC
6 );

1 INSERT INTO ventas (cliente_id, fecha, total)
2 SELECT
3     (random()*1000)::int,
4     CURRENT_DATE - (random()*365)::int,
5     (random()*1000)::numeric
6 FROM generate_series(1, 50000);

1 EXPLAIN ANALYZE
2 SELECT * FROM ventas
3 WHERE cliente_id = 10 AND fecha = CURRENT_DATE;

1 CREATE INDEX idx_ventas_cliente_fecha
2 ON ventas(cliente_id, fecha);
```

Preguntas:

- ¿Qué pasa si se consulta solo por fecha?
- ¿Importa el orden de las columnas?

Ejercicio 4: ORDER BY

```
1 EXPLAIN ANALYZE
2 SELECT * FROM ventas
3 ORDER BY fecha DESC LIMIT 10;

1 CREATE INDEX idx_ventas_fecha
2 ON ventas(fecha DESC);
```

Pregunta: ¿Desaparece la operación de ordenamiento?

Ejercicio 5: Índices parciales

```
1 CREATE INDEX idx_clientes_mendoza
2 ON clientes(email)
3 WHERE ciudad = 'Mendoza';

1 EXPLAIN ANALYZE
2 SELECT * FROM clientes
3 WHERE ciudad = 'Mendoza'
4 AND email = 'cliente10000@mail.com';
```

Ejercicio 6: LIKE

```
1 EXPLAIN ANALYZE
2 SELECT * FROM clientes
3 WHERE nombre LIKE 'Cliente 1%';

1 CREATE INDEX idx_clientes_nombre
2 ON clientes(nombre);

1 SELECT * FROM clientes
2 WHERE nombre LIKE '%1';
```

Pregunta: ¿Cuándo PostgreSQL usa el índice y cuándo no?

Ejercicio 7: Impacto en escritura

- Insertar datos sin índices y medir tiempo.
- Crear índices.
- Insertar nuevamente y comparar.

Ejercicio 8: Visualización del contenido de índices

Instalar la siguiente extensión:

```
1 CREATE EXTENSION pageinspect;
2 -- Visualización del contenido
3 SELECT *
4 FROM bt_page_items('idx_clientes_email', 1);
```

PARTE II

Ejercicio 1: Índice HASH básico

```
1 CREATE TABLE usuarios (  
2     id SERIAL PRIMARY KEY,  
3     email TEXT,  
4     nombre TEXT  
5 );  
  
1 INSERT INTO usuarios (email, nombre)  
2 SELECT  
3     'user' || i || '@mail.com',  
4     'Nombre ' || i  
5 FROM generate_series(1, 50000) i;  
  
1 CREATE INDEX idx_usuarios_email_hash  
2 ON usuarios USING HASH (email);  
  
1 EXPLAIN ANALYZE  
2 SELECT * FROM usuarios  
3 WHERE email = 'user1000@mail.com';
```

Preguntas:

- ¿Aparece Index Scan?
- ¿Qué diferencia hay con B-tree?

Ejercicio 2: Comparación HASH vs B-tree

```
1 CREATE INDEX idx_usuarios_email_btree  
2 ON usuarios(email);  
  
1 EXPLAIN ANALYZE  
2 SELECT * FROM usuarios  
3 WHERE email = 'user2000@mail.com';
```

Tarea:

- Comparar planes de ejecución
- ¿Cuál usa PostgreSQL?
- ¿Por qué?

Ejercicio 3: Caso donde HASH no sirve

```
1 EXPLAIN ANALYZE  
2 SELECT * FROM usuarios  
3 WHERE email > 'user40000@mail.com';
```

Preguntas:

- ¿Se usa el índice hash?
- ¿Por qué no?

Ejercicio 4: Otro ejemplo

```
1 CREATE TABLE productos (  
2     id SERIAL PRIMARY KEY,  
3     codigo TEXT,  
4     precio NUMERIC  
5 );  
  
1 INSERT INTO productos (codigo, precio)  
2 SELECT  
3     'COD' || i,  
4     (random()*1000)::numeric  
5 FROM generate_series(1, 30000) i;  
  
1 CREATE INDEX idx_productos_codigo_hash  
2 ON productos USING HASH (codigo);  
  
1 EXPLAIN ANALYZE  
2 SELECT * FROM productos  
3 WHERE codigo = 'COD15000';
```

Ejercicio 5: Impacto en inserciones

- Insertar datos sin índice
- Crear índice HASH
- Insertar nuevamente y comparar tiempos

```
1 INSERT INTO productos (codigo, precio)  
2 SELECT  
3     'NEW' || i,  
4     (random()*1000)::numeric  
5 FROM generate_series(1, 10000) i;
```

Ejercicio 6: Colisiones (conceptual)

Preguntas:

- ¿Qué es una colisión en un índice hash?
- ¿Cómo las maneja PostgreSQL?

Ejercicio 7: Desafío

```
1 SELECT * FROM usuarios  
2 WHERE email = ?;
```

Tarea:

- ¿Elegirías HASH o B-tree?
- Justificar la decisión

Conclusión

- HASH solo sirve para comparaciones por igualdad (=)
- B-tree es más flexible y generalmente preferido

PARTE III

Introducción a PGBENCH

pgbench es una herramienta incluida en PostgreSQL que permite medir el rendimiento de una base de datos mediante pruebas de carga y concurrencia.

Creación de la base de datos

Primero se crea una base de datos para las pruebas:

```
1 createdb prueba_bench
```

Inicialización del entorno

Se generan las tablas y datos de prueba:

```
1 pgbench -i prueba_bench
```

Para aumentar el tamaño de los datos:

```
1 pgbench -i -s 10 prueba_bench
```

Ejecución de un benchmark básico

Ejemplo de prueba con múltiples clientes:

```
1 pgbench -c 10 -t 100 prueba_bench
```

Parámetros:

- -c: cantidad de clientes concurrentes
- -t: cantidad de transacciones por cliente

Pruebas por tiempo

Ejecutar durante un tiempo determinado:

```
1 pgbench -c 10 -T 60 prueba_bench
```

Interpretación de resultados

La salida incluye métricas como:

- TPS (transacciones por segundo)
- Latencia promedio

Uso de scripts personalizados

Se puede definir un archivo con consultas propias.

Archivo `test.sql`:

```
1 SELECT * FROM pgbench_accounts WHERE aid = :aid;
```

Ejecutar:

```
1 pgbench -c 10 -T 30 -f test.sql prueba_bench
```

Evaluación de índices

Ejemplo de comparación:

Sin índice:

```
1 DROP INDEX IF EXISTS idx_aid;
```

Crear índice:

```
1 CREATE INDEX idx_aid ON pgbench_accounts(aid);
```

Luego ejecutar `pgbench` y comparar resultados.

Modo detallado

Para obtener estadísticas más precisas:

```
1 pgbench -c 5 -t 10 -r prueba_bench
```

Ejemplo completo

```
1 createdb bench_test
2 pgbench -i -s 5 bench_test
3 pgbench -c 20 -T 60 bench_test
```